

ERP Application – Full-Stack Developer Case Study

Industrial Manufacturing & Supply Chain Workflow System

End-to-End Business Flow: Enquiry → Quotation → Sales Order → Reservation → Dispatch

Candidate Name:	Bhargav Reddy
Role Evaluated:	Full-Stack Developer
Technology Stack:	PERN (PostgreSQL, Express.js, React.js, Node.js)
Submission Date:	September 17, 2026
Project Domain:	Industrial Manufacturing & Supply Chain ERP
Repository Scope:	Full Working Implementation, Tests & API Documentation

Table of Contents

- 1. Cover Page
- 2. Table of Contents
- 3. Introduction
- 4. Problem Statement
- 5. Project Objectives
- 6. Technology Stack
- 7. System Architecture
- 8. Database Design & Relational Schema
- 9. Application Modules
- 10. Complete Business Workflow
- 11. REST API Documentation
- 12. Authentication & Role-Based Access Control (RBAC)
- 13. Inventory Reservation & Concurrency Logic
- 14. Frontend Implementation & UI Design
- 15. Testing & Verification Results
- 16. Setup, Configuration & Startup Guide
- 17. Application Screenshots & Interface Walkthrough
- 18. Technical Challenges & Implemented Solutions
- 19. Assumptions & Limitations
- 20. Conclusion & Future Roadmap

3. Introduction & Business Context

In modern industrial manufacturing and B2B distribution environments, commercial transactions require strict coordination between sales negotiations, contract management, credit terms, and physical inventory control. Unlike consumer e-commerce where orders are paid instantly and stock is immediately subtracted, business-to-business transactions operate through multi-stage commercial workflows.

This Enterprise Resource Planning (ERP) application provides a robust, production-grade technical implementation of the complete sales-to-fulfillment lifecycle for an industrial supply company. Developed using the PERN technology stack (PostgreSQL, Express.js, React.js, and Node.js), the platform guarantees data integrity, exact monetary accuracy, row-level concurrency protection, and role-based administrative governance.

4. Problem Statement & Business Challenges

Industrial suppliers frequently suffer from fragmented communication between sales representatives and warehouse managers. This leads to severe business vulnerabilities:

- **Stock Over-Allocation & Overselling:** When multiple sales reps accept quotations against the same physical stock without synchronous inventory reservations, orders get confirmed that the warehouse cannot fulfill.
- **Floating-Point Drift in Quotations:** Relying on client-side or unsafe floating-point calculations results in billing errors across quantities, discounts, and multi-tier Goods and Services Tax (GST).
- **Loss of Audit Traceability:** In the absence of a relational schema, companies cannot trace customer enquiries to quotations, confirmed orders, and physical vehicle dispatches.
- **Unauthorized Administrative Actions:** Without backend-enforced RBAC, sales personnel can confirm orders or ship stock without credit or inventory authorization.

5. Project Objectives

- **Traceable Business Workflow:** Customer Enquiry → Quotation → Sales Order → Inventory Reservation → Dispatch.
- **Server-Side Calculation Authority:** Enforce exact financial calculations on Node.js using minor-currency units.
- **ACID Concurrency Protection:** Prevent double-reservation anomalies using SELECT ... FOR UPDATE row locks.
- **Role-Based Security:** Isolate permissions between ADMIN and SALES_USER at the Express API layer.
- **Demonstrable Clean UI:** Deliver a responsive React 18 administrative dashboard covering all 4 core screens.

6. Technology Stack

Component	Technology	Architectural Responsibility
Database Layer	PostgreSQL (v14+)	ACID relational persistence, constraints, foreign keys, row locks
Backend Runtime	Node.js (v18+) & Express	RESTful routing, business logic, transactions, JWT auth
Frontend Layer	React 18 + Vite	Single-Page Application, state management, responsive UI
Authentication	JWT + bcryptjs	Stateless bearer authorization, salt-hashed passwords (10 rounds)
Schema Validation	Zod	Type-safe runtime validation for incoming payloads and parameters
Automated Testing	Jest & Supertest	Automated test suites for financial math, RBAC, and race conditions
API Documentation	Swagger (OpenAPI 3.0)	Interactive API explorer hosted natively at /api-docs

7. System Architecture

The platform operates as a 3-tier decoupled architecture:

- 1. Presentation Layer (React 18 SPA):** Vite-bundled single-page application with Axios interceptors managing JWT tokens, real-time inventory polling, and interactive state.
- 2. Application Layer (Express REST API):** Centralized middleware pipeline enforcing Bearer authentication, RBAC authorization, Zod schema validation, and financial calculations.
- 3. Data Persistence Layer (PostgreSQL):** Normalized relational engine executing transactional locking (SELECT ... FOR UPDATE), constraint validations, and indexed queries.

8. Seed Industrial Products Master

Code	Product Name	Unit	Price	Physical	Reserved
IND-BRG-001	Industrial Bearing 6205-2RS	PCS	■450.00	200	60 (Avail: 140)
HYD-PMP-002	Hydraulic Pump HP-35	SET	■8,500.00	50	10 (Avail: 40)
STL-GAR-003	Steel Gear Assembly SGA-12	SET	■3,200.00	120	30 (Avail: 90)
CNV-BLT-004	Heavy Conveyor Belt 500mm	MTR	■1,250.00	300	50 (Avail: 250)
IND-MTR-005	Three-Phase Motor 5HP	PCS	■14,500.00	40	5 (Avail: 35)

PRS-VLV-006	High Pressure Valve PV-10	PCS	■1,800.00	150	20 (Avail: 130)
-------------	---------------------------	-----	-----------	-----	-----------------

13. Inventory Reservation & Concurrency Logic

Inventory reservation is the central business challenge of this case study. When an Admin confirms a Sales Order, the system must guarantee that stock cannot be double-allocated even under extreme concurrent load.

The Concurrency Challenge:

Consider available stock = 100 units. User A requests 80 units, and User B requests 50 units simultaneously. Without row locking, both requests read available = 100, both proceed to reserve, resulting in 130 reserved units against 100 physical units. This corrupts physical inventory integrity.

The PostgreSQL Row-Locking Solution:

Our implementation in backend/src/controllers/salesOrderController.js executes:

1. **Explicit Transaction:** Initiated via `await client.query('BEGIN')`.
2. **Consistent Lock Ordering:** Multi-item order product IDs are sorted in ascending order (`ORDER BY product_id ASC`). This guarantees that concurrent transactions acquire locks in identical sequence, completely preventing database deadlocks.
3. **Row-Level Exclusive Lock:** Executed via:
`SELECT product_id, physical_quantity, reserved_quantity, (physical_quantity - reserved_quantity) AS available_quantity FROM inventory WHERE product_id = ANY($1) ORDER BY product_id ASC FOR UPDATE;`
4. **All-or-Nothing Rule:** Every product is checked against `Available >= Required`. If any item is insufficient, the entire transaction rolls back immediately with `ROLLBACK` and returns `HTTP 409 Conflict`. No partial reservation is ever committed.
5. **Atomic Reservation:** If stock is sufficient, `UPDATE inventory SET reserved_quantity = reserved_quantity + $1` is executed, order status becomes 'CONFIRMED', and the transaction commits.

10. Dispatch Module Logic

When an Admin ships goods from the factory:

- Order must be in CONFIRMED status (cannot dispatch PENDING or CANCELLED orders).
- Transport details (Vehicle Registration Number & Driver Name) are recorded.
- In an atomic transaction, **both** Physical Quantity and Reserved Quantity decrease by the order quantity.
- Notice that Available Quantity remains unchanged: $Available = (Physical - Qty) - (Reserved - Qty) = Physical - Reserved$.
- Generates a unique dispatch tracking number (e.g. DSP-20260917-0001) and marks order DISPATCHED.

15. Automated Testing Suite & Verification

The application includes 13 automated tests covering all mandatory case study scenarios and bonus concurrency race conditions. All 13 tests passed successfully with 100% pass rate:

Test ID	Test Suite	Verification Objective	Result
TEST 1	Calculator Unit	Accurately calculates line item Base, Discount %, 18% GST, and Line Total	PASS
TEST 1b	Calculator Unit	Accurately calculates multi-item quotation grand totals	PASS
TEST 1c	Calculator Unit	Rejects negative prices, zero quantities, and out-of-bounds percentages	PASS
TEST 2	Business Logic	Enforces that DRAFT quotation cannot create a Sales Order	PASS
TEST 2b	Business Logic	Enforces that REJECTED quotation cannot create a Sales Order	PASS
TEST 3	Business Logic	Enforces that the same quotation cannot generate duplicate Sales Orders	PASS
TEST 4	Business Logic	Cannot reserve more than available inventory (Physical – Reserved)	PASS
TEST 4b	Business Logic	Atomic dispatch decreases physical and reserved stock correctly	PASS
TEST 5	Security / RBAC	Rejects unauthenticated requests with HTTP 401 Unauthorized	PASS
TEST 5b	Security / RBAC	Rejects SALES_USER from confirming Sales Orders with HTTP 403 Forbidden	PASS
TEST 5c	Security / RBAC	Rejects SALES_USER from processing dispatch with HTTP 403 Forbidden	PASS
TEST 5d	Security / RBAC	Rejects SALES_USER from creating products master with HTTP 403 Forbidden	PASS
BONUS	Concurrency Test	Simultaneous requests: User A (80 units) & User B (50 units) on 100 stock. Only 1 succeeds	PASS

16. Setup, Configuration & Startup Guide

Prerequisites: Node.js (v18+) and PostgreSQL (v14+) or a free cloud database (Neon/Supabase).

1. Database Setup: `cd backend && npm run db:setup` (Creates database, schema, and seeds)
2. Run Tests: `cd backend && npm test` (Executes all 13 automated tests)
3. Start Backend: `cd backend && npm run dev` (API runs on `http://localhost:5000`)
4. Start Frontend: `cd frontend && npm run dev` (React app on `http://localhost:5173`)

Test Credentials:

- Admin: admin / Admin@123 (Role: ADMIN)
- Sales User: sales / Sales@123 (Role: SALES_USER)

20. Conclusion

The Fundsroom PERN Stack ERP application fulfills all functional, architectural, and business workflow requirements specified in the case study. By prioritizing database consistency, transaction safety, and backend authorization over superficial UI animations, the system demonstrates the core engineering values required for enterprise software development.